

答辩临场攻略 — 基于真实答辩记录的针对性准备

整理了其他组答辩中老师提问的规律，逐一对照我们的项目准备回答。

一、老师提问规律总结

从其他组的答辩记录看，这位老师（13485342029）的提问有非常清晰的模式：

- 1. 程序必须跑起来** — 第一位同学程序启动失败，老师立刻转向核心算法逻辑的追问。所以你必须确保 `npm run dev` 能正常启动。
- 2. 追问算法实现细节** — 快排是不是双指针？冒泡有没有提前终止优化？迷宫状态快照是全量还是增量？这些不是问概念，是问“你是怎么实现的”。
- 3. 追问设计决策的 WHY** — 为什么用 txt/json/markdown 三种格式？为什么选了 5 层架构？为什么用合并定时器？老师要听的是“你的考量”，不是“你做完了什么”。
- 4. 演示出错 = 口头追问** — 哈夫曼树展示出问题，老师直接转为口头问选择排序的核心思想。所以如果演示挂了，不要慌，用嘴说清楚。
- 5. 喜欢追问边界情况** — 负边权拦截、50 个数据上限防密集、随机数据规则。老师关心的是“你的系统能不能处理异常输入”。

二、针对我们项目的逐类准备

A. 算法核心逻辑类

如果老师问：冒泡排序有提前终止的优化吗？

有的。我们的冒泡排序实现了 `swapped` 标志位：内层循环如果一整轮没有发生任何交换，说明数组已经有序，直接 `break` 退出。这个优化在 `templateCode` 里有体现，步骤生成器也会在提前终止时输出“本轮无交换发生，已完全有序”的描述。不只是冒泡——快排的递归在子数组长度 ≤ 1 时也会提前返回。

如果老师问：快排是双指针还是单指针？

我们的快排实现用的是 Lomuto 分区方案——单指针 `i` 记录“小于 `pivot` 的区域的末尾”，`j` 遍历数组。选择这个方案的原因是它的步骤可视化更清晰：`i` 和 `j` 的移动方向一致，学生容易理解“小的放左边，大的自然留在右边”。双指针（Hoare 方案）虽然交换次数少，但两个指针相向移动动画不太直观。三指针还需要额外处理基准位置的占位，反而增加理解负担。

如果老师问：Dijkstra 能处理负边权吗？如果有负边权会怎样？

Dijkstra 不能处理负边权——它基于贪心策略，一旦节点被标记为 `visited` 就不再更新。如果出现负边权，节点可能在 `visited` 之后被更短的路径更新，但 Dijkstra 不会再处理它，导致结果错误。在我们的图自定义输入里，用户可以输入负权重，但 `GraphVisualizer` 不会阻止——这是教学目的：让学生亲自看到 Dijkstra 在负边权图上的错误结果，理解算法的适用边界。不过如果老师问到，可以说“后续可以加一个负边权警告提示”。

如果老师问：迷宫的生成和求解是怎么做的？

迷宫生成用的是 Randomized DFS（递归回溯算法）——从边界一个单元格出发，随机选择四个方向中尚未访问的，打通墙壁前进。当四个方向都走不通时回溯。这个算法生成的迷宫有一个重要特性：任意两单元格之间只有唯一路径（完美迷宫）。迷宫求解我们实现了两种——BFS 求最短路径，DFS 深度搜索（不保证最短但能展示回溯过程）。每一步的网格状态通过 `gridData` 字段传递给 `GridVisualizer`。

B. 设计决策 WHY 类

如果老师问：为什么选择 5 层分层架构？

核心目标是“高内聚、低耦合”。55 个算法和 5 种可视化组件如果不分层，代码会变成一锅粥——改排序算法可能破坏树的渲染。我设计的 5 层中，每一层只依赖下一层：算法引擎层完全不 `import React`，它是纯 TypeScript 函数，输入数组、输出步骤序列。这意味着算法逻辑可以独立测试，换了 UI 框架也不影响。当时参考了 OSI 网络模型的分层思想——每层有明确的职责边界。

如果老师问：为什么用合并定时器而不是给每个算法独立的播放器？

我们只有一个全局播放引擎，在 Zustand 里用 `setInterval` 统一管理。好处是：所有算法共享同一套播放控制（Space 播放/暂停、`←` `→` 单步、R 重置），速度调节也只需要改一个 `interval`。如果每个算法独立一个播放器，状态同步会很复杂——特别是在我们加了 5 档速度调节之后。

如果老师问：为什么快照是全量保存而非增量？

全量快照的核心优势是“单步后退”。如果是增量——比如只记录“交换了位置 2 和位置 3”——后退时需要用逆操作来还原。但对于插入排序、BST 插入这类操作，逆操作不是简单的反操作，需要额外维护 `undo` 栈。全量快照直接读 `steps[currentStep - 1]`，一步到位。代价是内存占用——55 个算法平均 50 步，每步存一个数组，总共也就几千个数字，完全在可接受范围。

C. 随机数据与边界处理类

如果老师问：随机数据是怎么生成的？为什么不能全用同一种随机？

不同算法的输入约束完全不同。如果全部用 `Array(8).fill(0).map(() => random(1,99))`：N 皇后会生成 `[23,67,15]` 这种无意义的棋盘大小；二分查找会生成无序数组导致查找失败；中缀表达式会生成 `" ;Ca "` 这种垃圾 ASCII。所以我为 55 个算法分别定制了生成规则。核心原则是“随机但不随便”——每条规则保证生成的测试数据对该算法合法可用。

如果老师问：数据上限是多少？为什么设这个上限？

排序类算法限制在 8 个元素左右（随机生成时），树类限制在 7-15 个节点。主要是为了可视化效果——柱形图超过 15 根会非常拥挤，树超过 5 层（31 个节点）会在画布上溢出。用户手动输入没有做硬性限制，但步骤数会相应增加。后续可以加一个输入长度校验。

D. UI 与技术选型 WHY 类

如果老师问：为什么用黑底（暗色主题）？

两个原因。一是 IDE 风格的暗色主题让代码面板的语法高亮更加醒目——紫色、绿色、橙色在深底上对比度远高于白底。二是长时间看算法动画不刺眼，适合课堂投影。不过这个设计只是 CSS 变量控制的，后续可以加亮色主题切换，不需要改组件代码。

如果老师问：为什么自研语法高亮器而不用成熟的库？

零依赖 + 完全可控。我们的代码面板只展示算法模板代码（已知的 TypeScript 代码，不是任意用户输入），语法高亮的需求非常确定——8 类 token 就够了。自研的 tokenizeLine 只有约 60 行代码，识别关键字/字符串/数字/注释/操作符/类型/变量/标点。如果引入 Prism.js 或 highlight.js，会增加约 100KB 的依赖体积，而且它们的 tokenizer 需要额外的配置和主题文件。对于我们的场景，60 行代码完全够用。

E. 可视化实现细节类

如果老师问：迷宫回溯的状态快照存储了什么？

每一步的 gridData 是一个完整的二维数组快照 (`maze.map(row => [...row])`)，记录了每个单元格的当前状态。1 表示通路，0 表示墙壁。额外用 gridCells 字段标记特定单元格的视觉状态——visiting 表示正在挖掘（黄色），visited 表示已挖掘（蓝色），path 表示最终路径（绿色）。这和 VisualStep 的全量快照原则一致。

如果老师问：树布局为什么不用现成的算法库？

我调研过 d3-hierarchy，但它假设的是“给定父子关系的树数据”，而我们的树是以层序数组存储的（index i 的左子=2i+1，右子=2i+2），且有空节点（值=0）需要跳过。d3 的布局算法不能直接套用。最终自己实现了递归子树区域分配算法——每个节点把水平空间分给左右子树，无论树是平衡还是偏斜都能自然展开。

三、高概率临场问题 TOP 5

根据其他组答辩记录，这 5 个问题出现概率最高：

TOP 1：你的 XX 算法是怎么实现的？（核心逻辑追问） → 别说概念，说实现细节。比如“冒泡排序的 generateSteps 函数中，外层循环 i 从 0 到 n-2，内层 j 从 0 到 n-2-i，每一步比较输出一个 compare 步骤，交换输出一个 swap 步骤”。

TOP 2：你为什么这样设计？（设计决策 WHY） → 永远准备一个“对比”——“我对比了 A 和 B，选了 B，因为……”。比如 Zustand vs Redux、全量快照 vs 增量、自研高亮 vs Prism.js。

TOP 3：如果输入了异常数据怎么办？（边界处理） → “目前没有硬性限制，但后续可以加。随机生成是有规则约束的。手动输入目前信任用户。”

TOP 4：这个功能的 AI 参与了多少？（AI 相关） → “AI 生成了框架代码，我做了 XX 修改。最终修改比例约 20%。” 强调人工判断力。

TOP 5：演示出问题了——直接口头回答！（应急） → “老师，这个功能的核心逻辑是这样的……（口头描述）。演示的问题我记录一下，会后排查。” 不要慌，展现工程素养。

四、演示前检查清单

- ☐ `npm run dev` 能正常启动
- ☐ 冒泡排序默认数据能正常播放到结束
- ☐ 空格键、方向键功能正常
- ☐ 准备好几个演示数据在剪贴板：
 - 排序：90,10,50,30,70,20,60,40

- 树: 50,30,70,20,40,60,80
- 图: 4,0,1,5,0,2,2,1,3,1
- N皇后: 8
- ☐ VS Code 打开项目, 方便被要求看代码
- ☐ 答辩准备 .md 文件在手机上备一份 (以防电脑出问题还能看手机说)
- ☐ 深呼吸, 别紧张——你做的比他们都多